



WeCloudData

Reshaping and Pivoting Data

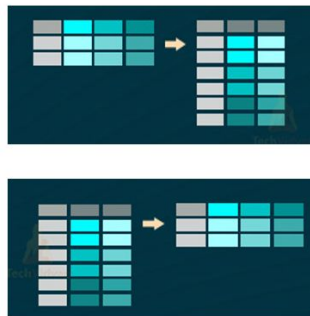
Reshape It! `stack` · `unstack` · `pivot_table`



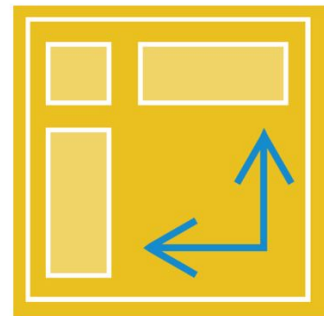


- Why does your data keep changing shape?
- MultiIndex
- Functions

Reshaping data



Pivot Table



Agenda.



WeCloudData

Data Reshaping



Why Reshape Data?

Why

Because the best shape depends on what you want to do next.

Raw log view

date	region	sales
Jan	East	100
Jan	West	80
Feb	East	110
Feb	West	95



Report view

date	East	West
Jan	100	80
Feb	110	95

Same values. Different layout.

Long format is easier to **collect and filter**.

Wide format is easier to **compare side by side**.

Reshaping is the bridge between data storage, analysis, plotting, and reporting.



Long vs. Wide Format

Data Shapes

Before choosing a function, identify the current shape and the target shape.

LONG / TALL

date	person	message
Mon	Ali	Hi
Mon	Sara	OK
Tue	Ali	Done
Tue	Sara	Check



reshape

WIDE / MATRIX

date	Ali	Sara
Mon	Hi	OK
Tue	Done	Check

- One observation per row
- Great for logging, filtering, and groupby
- Can feel hard to scan side by side

- One entity becomes a column
- Great for comparison, correlation, and many plots
- New categories create new columns



WeCloudData

MultiIndex



What is MultiIndex

What is MultiIndex

MultiIndex is an index with more than one "level" - like a folder inside a folder.

Example: rows labeled by (year, month) instead of just one thing.

year	month	value
2023	Jan	10
	Feb	12
2024	Jan	14
	Feb	15



Where Does MultiIndex Come From?

What is MultiIndex

It often appears after `groupby()` or `pivot_table()`.

year	month	value
2023	Jan	10
2023	Feb	12
2024	Jan	14
2024	Feb	15



```
df.groupby(['year', 'month'])['value'].sum()
```

year	month	value
2023	Jan	10
	Feb	12
2024	Jan	14
	Feb	15

MultiIndex = more than one row-label level. Here, year is the outer level and month is the inner level.



unstack(): Move a Row Level to Columns

unstack()

Use it when you already have a MultiIndex and want a wider table.

BEFORE: MultiIndex rows

year	month	value
2023	Jan	10
	Feb	12
2024	Jan	14
	Feb	15



AFTER: month becomes columns

year	Jan	Feb
2023	10	12
2024	14	15

```
df.unstack('month')
```

Default behavior: unstack() moves the innermost row level to columns.



unstack() Can Create Missing Cells

unstack()

If one row-column combination does not exist, pandas fills it with NaN.

year	month	value
2023	Jan	10
2023	Feb	12
2024	Jan	14



year	Jan	Feb
2023	10	12
2024	14	NaN

```
df.unstack(fill_value=0)
```

Use fill_value only when the replacement makes sense. For sales count, 0 may be okay. For temperature, 0 may be misleading.



stack(): Move Columns Back to Rows

stack()

Use it when your table is too wide and you need a tidy long format.

BEFORE: wide table

date	East	West
Jan	100	80
Feb	110	95



AFTER: longer table

date	region	sales
Jan	East	100
Jan	West	80
Feb	East	110
Feb	West	95

```
wide_df.stack()
```

stack() is the mirror idea of unstack(): columns are folded down into the row index.



When Should You Use `stack()`?

`stack()`

Most often, `stack()` helps clean wide spreadsheet-style data.

country	2021	2022	2023
A	50	53	58
B	40	42	47



country	year	value
A	2021	50
A	2022	53
A	2023	58
B	2021	40

- Good for trend analysis
- Good before groupby/filtering/modeling
- Good when column names are really values

```
df.set_index('country').stack()
```



stack() and unstack() Are Mirrors — Mostly

stack() and unstack()

They usually preserve the same data, but not always the same shape details.

```
df.stack().unstack()  
# usually returns something very close to  
df
```

- NaNs can appear when combinations are missing
- Index/column order can change
- Names of levels matter for readability



Debugging habit: after every reshape, check `.head()`, `.shape`, `.index.names`, and `.columns`.



WeCloudData

Raw Records to Summary Tables



`pivot(): Reshape Without Aggregation`

`pivot()`

Use it when each row-column pair has exactly one value.

BEFORE: long records

date	region	sales
Jan	East	100
Jan	West	80
Feb	East	110
Feb	West	95



AFTER: wide table

date	East	West
Jan	100	80
Feb	110	95

```
df.pivot(  
    index='date',  
    columns='region',  
    values='sales'  
)
```

Key idea: `pivot()` only rearranges existing values. It does not sum, average, or count them.



When pivot() Breaks

pivot()

If one output cell has multiple records, pivot() does not know which value to place there.

PROBLEM: Jan + East appears twice

date	region	sales
Jan	East	100
Jan	East	40
Jan	West	80
Feb	East	110
Feb	West	95



date	East	West
Jan	?	80
Feb	110	95

Which value goes into Jan–East?

```
df.pivot(index='date', columns='region',  
          values='sales')  
# ValueError: duplicate entries cannot reshape
```

This is where we need aggregation. The next tool, `pivot_table()`, can reshape and aggregate at the same time.



`pivot_table()`: Build a Summary Table

`pivot_table()`

Start with raw records. Choose rows, columns, values, and aggregation.

date	region	sales
Jan	East	100
Jan	East	40
Jan	West	80
Feb	East	110
Feb	West	95



date	East	West
Jan	140	80
Feb	110	95

Why `pivot_table()`? We need one summary value for each date–region cell. Jan + East has multiple records, so we aggregate them with `sum`.

```
df.pivot_table(  
    index='date',  
    columns='region',  
    values='sales',  
    aggfunc='sum'  
)
```



Anatomy of pivot_table()

`pivot_table()`

Map each argument to the final table.

```
df.pivot_table(  
    index='date',      # new rows  
    columns='region',  # new columns  
    values='sales',    # cell values  
    aggfunc='sum'      # aggregate multiple records  
)
```

argument	question it answers
index	What should define each row?
columns	What should spread across columns?
values	What number/text fills the cells?
aggfunc	How should multiple values be aggregated?

Default aggfunc is mean. For business reports, be intentional: sum, count, mean, max, etc.



pivot() vs. pivot_table()

pivot() vs. pivot_table()

The key difference is whether aggregation is needed.

function	when it works	repeated pair?
<code>pivot()</code>	Simple reshape only	Raises an error
<code>pivot_table()</code>	Summary table	Aggregates values

Rule of thumb: use `pivot()` for a simple reshape; use `pivot_table()` when one output cell may need aggregation.

```
# two rows for Jan + East
# pivot() cannot choose one value for the
cell
```

`pivot_table()` is safer for real-world summary tables because real data often has repeated row-column combinations.



aggfunc Controls the Meaning of the Table

aggfunc

The same raw data can answer different business questions.

question	aggfunc	example meaning
Total sales by region	sum	How much did we sell?
Average order size	mean	What is a typical order?
Number of orders	count	How many transactions?
Best month	max	What is the peak value?

```
df.pivot_table(  
    index='date', columns='region',  
    values='sales',  
    aggfunc=['sum', 'mean', 'count']  
)
```

Multiple aggfuncs create another column level. That is useful, but it can make the result harder to read.



Manual Route: `set_index()` + `unstack()`

Pro Combo

Useful when you want more control than a one-line `pivot_table`.

```
df.set_index(['date',  
'region'])['sales'].unstack('region')
```

This creates a MultiIndex first, then moves region from row level to columns.

Use when you already know exactly which columns should become index levels
Useful for multiple value columns or more customized reshaping
But for most summary tables, `pivot_table()` is easier to read

simple report?

Use `pivot_table()`

precise index control?

Use `set_index()` + `unstack()`



Which tool to reach for?

Goals and Tools

current situation	goal	tool
Already have MultiIndex rows	Move one row level to columns	<code>unstack()</code>
Wide table	Make it longer / tidy	<code>stack()</code>
Raw long records	Create a summary table	<code>pivot_table()</code>
No duplicates, simple reshape	Just spread values across columns	<code>pivot()</code>
Need exact level control	Build levels manually	<code>set_index() + unstack()</code>



Which tool to reach for?

Mini Case 1

Time Series: one row per variable

Got stock prices logged as (date, ticker, price)?

- `unstack('ticker')` turns it into one row per date and one column per stock
- It's perfect for computing correlations or plotting multiple lines at once.

date	ticker	price
Jan 1	AAPL	190
Jan 1	MSFT	420
Jan 2	AAPL	192
Jan 2	MSFT	415



date	AAPL	MSFT
Jan 1	190	420
Jan 2	192	415

```
prices.unstack('ticker')
```

Compare tickers side by side
Plot multiple lines
Compute correlations



Which tool to reach for?

Mini Case 2

Messy Spreadsheet Exports

Excel exports often come "wide" with years as columns (2021, 2022, 2023...). To analyze trends over time, `stack()` turns those year-columns into a tidy "year" column, ready for `groupby`, filtering, modeling.

city	2021	2022	2023
Riyadh	120	135	150
Jeddah	80	91	100



city	year	value
Riyadh	2021	120
Riyadh	2022	135
Riyadh	2023	150
Jeddah	2021	80

```
df.set_index('city').stack()
```

After stacking, year becomes data instead of being hidden inside column names.



Common Mistakes

Common Mistakes

1

Mixing up rows and columns

Print `df.index` and `df.columns` before reshaping.

2

Using `pivot()` with duplicates

Use `pivot_table()` when repeated pairs exist.

3

Unnamed MultiIndex levels

Name levels so `level="month"` works.

4

Blindly filling NaNs with 0

Only fill missing values when 0 really means no value.

```
df.head()
df.shape
df.index.names
df.columns
```

Use these after each reshape step.



Cheat Sheet

Goals and Tools

Function	Direction	Best for	Basic Syntax
<code>unstack()</code>	row level → columns	MultilIndex to wide	<code>df.unstack("level")</code>
<code>stack()</code>	columns → row level	wide to long	<code>df.stack()</code>
<code>pivot()</code>	long → wide	simple reshape, no duplicates	<code>df.pivot(index, columns, values)</code>
<code>pivot_table()</code>	long → wide + aggregation	summary table	<code>df.pivot_table(index, columns, values, aggfunc)</code>
<code>set_index()+unstack()</code>	manual levels → wide	extra control	<code>df.set_index([...]).unstack("col")</code>